

Gymnázium Velké Meziříčí

Informační a komunikační technologie



Prší v jazyce JavaScript

ZÁVĚREČNÁ PRÁCE

Prohlašuji, že jsem svoji závěrečnou práci napsal samostatně a výhradně s použitím citovaných zdrojů.

Ve Velkém Meziříčí dne 10. 4. 2024

Jan Kroulík

Obsah

1	Úvod.....	4
2	Teoretická část.....	5
2.1	<i>Programovací jazyky.....</i>	<i>5</i>
2.1.1	Co jsou to programovací jazyky.....	5
2.1.2	Nižší programovací jazyky	5
2.1.3	Vyšší programovací jazyky.....	5
2.1.4	HTML.....	6
2.1.5	JavaScript	6
2.1.6	Knihovna p5.js	7
2.1.7	Visual Studio Code	7
2.2	<i>Prší.....</i>	<i>8</i>
2.2.1	Základní pravidla	8
2.2.2	Speciální karty a přídatná pravidla	8
3	Praktická část.....	9
3.1	<i>Příprava Visual Studio Code.....</i>	<i>9</i>
3.1.1	Instalace rozšíření.....	9
3.1.2	Vytvoření projektu p5.js.....	10
3.1.3	Spuštění lokálního serveru	11
3.2	<i>Třída Card.....</i>	<i>12</i>
3.2.1	Popis třídy a constructor().....	12
3.2.2	show()	13
3.2.3	setPos(), flip()	15
3.2.4	clicked()	15
3.2.5	use()	16
3.3	<i>Třída Button</i>	<i>16</i>
3.3.1	Popis třídy a constructor().....	16
3.3.2	setFunction(), setImage(), activate()	17
3.3.3	show(), clicked()	18
3.4	<i>Hlavní část sketch.js</i>	<i>19</i>
3.4.1	Základní proces hry	19
3.4.2	Balíčky a karetní akce	20
3.4.3	mouseClicked().....	21
3.4.4	Protihráč.....	22
3.4.5	Konec hry, restart	24
3.4.6	Grafická podoba	24
3.4.7	Zvukové efekty a hudba	25
3.4.8	Textový font.....	26
3.5	<i>Nahrávání na server.....</i>	<i>26</i>
4	Závěr	27
5	Seznam zdrojů	28
6	Seznam obrázků	29

1 Úvod

Když jsem v roce 2017 začínal s programováním, tehdy v jazyce Java, velmi mě bavilo vytvářet napodobeniny různých jednoduchých her, ať už se jednalo o Minesweeper nebo třeba klasického Hada. Jednou z her, kterou jsem se snažil naprogramovat, bylo karetní prší. Bohužel se mi nedařilo a tento projekt zůstal i přes několik pokusů stále nedokončený.

Vzpomněl jsem si na něj při výběru tématu závěrečné práce v semináři informatiky. Napadlo mě, že bych tím konečně mohl zpracovat projekt, na který jsem se těšil, a zároveň se posunout v programování trochu dál a něco nového se naučit. Tvorba hry se také dá vhodně skloubit s mojí další zálibou – tvorbou hudby. Ta bude ve hře zastoupena několika skladbami.

Teoretická část práce pojednává o základním rozdělení programovacích jazyků a jejich využití, dále rozebírá jednotlivé nástroje a jazyky, které budu ke své práci používat. Také přibližuje základní i rozšířená pravidla karetní hry prší.

Cíl praktické části je tvorba kódu hry prší v programovacím jazyce JavaScript, která bude nahrána na web a její hlavní náplní bude partie prší proti algoritmu počítače, schopnému být obstojným soupeřem. Hra bude doplněna o vlastní zvukové efekty, grafické zpracování jednotlivých elementů a vlastní hudbu.

Jazyk, ve kterém je kód hry vytvořen je JavaScript, na který jsem narazil díky tutoriálovému YouTube kanálu Daniela Shiffmana The Coding Train¹. Program využívá také knihovnu p5.js, na jejímž vývoji se Shiffman podílí. Tento jazyk i knihovnu jsem si oblíbil, příjemně se mi s nimi pracuje. Jako vývojové prostředí budu používat Visual Studio Code, mám s ním v souvislosti s prací v JavaScriptu dobré zkušenosti. Obrázky budu vytvářet v programu Malování a hudbu v programu FL Studio 21, se kterými jsem také zvyklý pracovat.

¹ The Coding Train. Online. 2024. Dostupné z: <https://www.youtube.com/@TheCodingTrain>. [cit. 2024-04-09].

2 Teoretická část

2.1 Programovací jazyky

2.1.1 Co jsou to programovací jazyky

Programovací jazyky jsou prostředkem k vytváření algoritmů, které mohou být provedeny na počítači. Jsou prostředníkem mezi programátorem a počítačem – programátor formuluje pomocí programovacího jazyka postup, kterým má být řešen určitý problém, výsledný program je poté převeden do strojového kódu a předán počítači, který ho interpretuje a udělá to, co je po něm žádáno. Programovací jazyky se dají rozdělit podle míry abstrakce (míra rozdílnosti mezi jazykem a strojovým kódem) na nižší a vyšší programovací jazyky.

2.1.2 Nižší programovací jazyky

Označení nižší programovací jazyk znamená, že jazyk poskytuje pouze malou nebo téměř žádnou abstrakci, žádný rozdíl mezi tím, jak daný jazyk funguje a jak funguje procesor počítače. Nižší programovací jazyky jsou snadné pro převedení na strojový kód, ale to za cenu toho, že jsou kvůli nízké abstrakci mnohem obtížněji osvojitelné než vyšší programovací jazyky. Efektivní práce s nimi vyžaduje podrobnější ponětí o fungování počítačového hardwaru. Mezi nižší programovací jazyky můžeme zařadit například jazyky C, Rust nebo Pascal.

2.1.3 Vyšší programovací jazyky

Na rozdíl od nižších programovacích jazyků, tato skupina jazyků poskytuje vyšší míru abstrakce, tedy větší přiblížení zapsaného kódu k tomu, jak by řešení problému zpracovával člověk. Měly by tedy být člověku lépe srozumitelné a jednodušší na představu, co po počítači požaduje. Tím se sníží počet programátorských chyb a zmenší čas potřebný k vytvoření algoritmu. Kvůli jejich odchylce od strojového kódu bývají ovšem pomalejší a převod do strojového kódu trvá delší dobu. Vyššími programovacími jazyky jsou například Java, Python, C++ nebo JavaScript.²

² Programovací jazyky. Online, prezentace. Praha: ČVUT, 2017. Dostupné z: https://www.fd.cvut.cz/personal/jerabem1/14prg/14PRG_03_programovaci_jazyky.pdf. [cit. 2024-04-09].

2.1.4 HTML

HTML (Hypertext Markup Language) je značkovací jazyk, který se používá ke tvorbě webových stránek, které jsou propojeny hypertextovými odkazy. Jazyk byl vytvořen v roce 1990 jako jednodušší varianta do té doby používaných jazyků, například TeX nebo PostScript. Současně vznikl i první web, který sloužil jako informační systém pro CERN.

Jazyk je charakterizován pomocí tzv. tagů (například tagy `<p>` a `</p>` značí začátek a konec odstavce textu) a jejich vlastností (atributů). Mezi tagy se uzavírají části textu a tím se určuje význam daného textu. Názvy tagů se uzavírají mezi znaky „<“ a „>“ a dělí se na párové a nepárové. Zatímco párové tagy nemůžou existovat bez párového ukončovacího tagu (před název tagu se přidá znak „/“, například `<body>` a `</body>`), nepárové tagy koncový tag nepoužívají a nepíše se za ně žádný text (například tag `<br>` slouží k vložení nového řádku).

Hojně se v jazyce HTML využívá i spouštění částí kódu nebo celých programů napsaných například v jazyce JavaScript. To je umožněno pomocí párového tagu `<script>`, mezi který je kód napsán a jazyk HTML ho vidí jako program místo čistého textu. Pro spuštění vlastního programu připojujeme k úvodnímu tagu `<script>` atribut `src`, který obsahuje lokaci souboru.

Soubory HTML kódu mají příponu `.html` nebo `.htm` a jsou spouštěny až v prohlížeči uživatele. Při spuštění webové stránky je nutné, aby alespoň jeden HTML soubor nesl název `index.html`. Tento soubor bude prvním, který bude spuštěn a z něho se uživatel může dostat k dalším stránkám.

2.1.5 JavaScript

JavaScript je objektově orientovaný programovací jazyk, využívaný nejčastěji jako součást HTML kódu stránky. Objektově orientované programování (OOP) spočívá v tom, že jednotlivé prvky jsou v programu seskupeny do entit (objektů), které jsou instancemi určité třídy (například objekt `Pes` je instancí nadřazené třídy `Zvíře`). Jednotlivé objekty dané třídy mají určité parametry a určité funkce, metody (například `Pes` a `Slepice` – objekty třídy `Zvíře` – mohou mít parametry `výška`, `hmotnost`, `počet nohou`, `barva` atd., mezi jejich metody může patřit například `chůze` nebo `vydání zvuku`).

Syntaxe jazyka JavaScript je podobná syntaxi jazyků C, C++ nebo Java, ale liší se ve využití. Na jeho základě vznikly další jazyky, například ActionScript, TypeScript nebo Node.js.

2.1.6 Knihovna p5.js

Knihovna p5.js je tzv. framework pro JavaScript. Je to knihovna obsahující předem naprogramované metody a nástroje, které slouží ke snadnějšímu programování zahrnující vizuální prvky, obrázky a grafický design. Knihovna se dá stáhnout v podobě souborů z adresy <https://p5js.org/download/> a poté zahrnout v HTML programu pomocí tagu `<script>`.

Pomocí této knihovny je umožněno využití mnoha funkcí, hlavně funkce `preload()`, `setup()` a `draw()`. Metoda `preload()` je spuštěná ještě před načtením stránky a slouží k přednačení určitých parametrů, k deklaraci určitých proměnných. Podobně funguje metoda `setup()`, která je ovšem spuštěna až po úplném načtení stránky. Slouží například k prvotnímu deklarování důležitých proměnných nebo spuštění úvodních metod, které mají být spuštěny pouze jednou. Poté se spustí metoda `draw()`, která se spouští pořád dokola, dokud program běží. Tato metoda se používá přímo pro vlastní dynamický program, například vykreslení jednotlivých snímků animace, protože probíhá pořád znovu.

2.1.7 Visual Studio Code

K úpravě jednotlivých souborů se používají textové editory, mezi nejzákladnější patří například Poznámkový blok v operačním systému Windows. Pro zrychlení a usnadnění programování a orientace v programu ale existují pokročilejší textové editory, umožňující například barevné odlišení jednotlivých částí kódu, automatické doplňování (například párových HTML tagů), formátování nebo vyhledávání chyb. Mezi takové editory patří i editor Visual Studio Code (VSC), vyvíjený společností Microsoft. Tento editor umožňuje programování v mnoha jazycích, například C, C#, C++, Java, Python, JavaScript nebo HTML. Editor umožňuje také využití mnoha rozšíření vyvíjených samotnými uživateli, která ještě více usnadňují programování v určité oblasti. Mezi taková rozšíření patří například p5.vscode, umožňující automatické založení a stažení knihovny p5.js a vytvoření p5.js projektu, nebo rozšíření Live Server, které vytvoří lokální server, který simuluje reálné webové stránky a umožňuje jednoduché vyzkoušení vzhledu a chování stránek bez nutnosti opatření vlastní adresy.

2.2 Prší

2.2.1 Základní pravidla

Prší je karetní hra pro více hráčů, známá také pod názvy mau-mau nebo pony. Obvykle se hraje s mariášovými kartami – 32 nebo 36 karet rozdělených do čtyř barev. Je možné ale hrát i s francouzskými (pokerovými) kartami, které obsahují 52 karet rozdělených do čtyř barev. Pravidla hry jsou v základu následující: balíček karet se zamíchá a každému hráči je rozdán stejný počet karet. Poté je doprostřed stolu položena jedna karta z balíčku, která slouží jako základní karta odhazovacího balíčku. Hráč, který je na tahu, musí odehrát vždy jednu ze svých karet, a to tak, že hodnota odehrané karty nebo její barva se shoduje s horní kartou v odhazovacím balíčku, například pokud je předchozí odehraná karta srdcová sedmička, musí hráč odehrát buď kartu s hodnotou sedm, nebo kartu s barvou srdce. Pokud hráč nemá žádnou hratelnou kartu, musí si tzv. líznout – vzít si vrchní kartu z dobíracího balíčku, otočeného rubem vzhůru. Následuje tah dalšího hráče.³

2.2.2 Speciální karty a přídatná pravidla

Pravidla hry nejsou všeobecně ustálená a univerzální, existuje proto mnoho různých verzí, jak se dá hrát. Jednou z možných změn je zavedení tzv. speciálních karet, které mají při odehrání určitý efekt. Jedná se například o karty, které nutí oponenta k přeskočení svého tahu (tzv. stání), nebo si musí líznout určitý počet karet.

Podle pravidel, která se používají v mém okruhu známých a přátel, se hraje s 52 pokerovými kartami a jako speciální karty zavádí eso, sedmičku a dámu. Pokud je zahráno eso, oponent přeskočí svůj tah, pokud ovšem on sám nezahraje eso. V takovém případě se efekt esa posouvá na dalšího hráče, pro kterého platí stejná podmínka. Pokud je zahrána sedmička, oponent si musí líznout dvě karty z lízacího balíčku. Pokud ale on sám odehraje sedmičku, efekt se posouvá dál, ale akumuluje se počet lízaných karet. Pokud tedy oponent zahraje sedmičku, další hráč si už nelízá dvě karty, ale čtyři, potom šest a osm. Poslední speciální kartou podle těchto pravidel je dáma, která se dá odehrát na jakoukoliv kartu – nemusí se shodovat hodnota ani barva. Odehrání dámy umožňuje hráči změnit barvu a oponent musí odehrát pouze kartu s touto barvou. Pokud tedy například hráč odehraje dámu a změni barvu na piky, hráč na tahu musí odehrát pikovou kartu.

³ Jak se hraje prší? Online. Justit.cz. 2023. Dostupné z: <https://justit.cz/2023/05/10/jak-se-hraje-prsi/>. [cit. 2024-04-09].

3 Praktická část

3.1 Příprava Visual Studio Code

Po předchozích dobrých zkušenostech jsem se rozhodl praktickou část této práce vytvořit v programovacím prostředí *Visual Studio Code (VSC)*. Ještě před samotným programováním je potřeba připravit všechny nástroje, které budeme potřebovat.

3.1.1 Instalace rozšíření

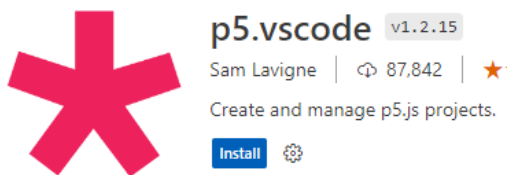
Pro usnadnění programování budeme používat rozšíření *p5.vscode* a *Live Server*. *p5.vscode* nám umožní automaticky založit projekt používající knihovnu *p5.js* a všechny náležité konfigurační soubory. Rozšíření *Live Server* vytvoří lokální server na naší IP adrese, který je možné použít k rychlému testování napsaného kódu místo toho, abychom upravený program neustále nahrávali na online server. Na ten nahrajeme pouze již finální verzi, kterou otestujeme na našem lokálním serveru. V praxi je možné tento krok přeskočit, toto rozšíření používám pouze pro usnadnění a urychlení celého procesu tvorby programu.

Tato rozšíření je ale potřeba nejdříve stáhnout. Po spuštění VSC tedy otevřeme záložku *Extensions* (rozšíření) v levém sloupci. Záložka se nachází na poslední pozici, má ikonu čtyř kostiček.



Obrázek 1: levý sloupec se záložkami ve VSC

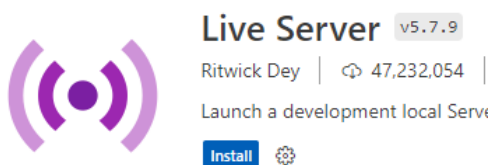
Ve vyhledávacím řádku zadáme název *p5.vscode* a klikneme na vyhledané rozšíření.



Obrázek 2: rozšíření p5.vscode

Klikneme na tlačítko *Install* a tím nainstalujeme rozšíření.

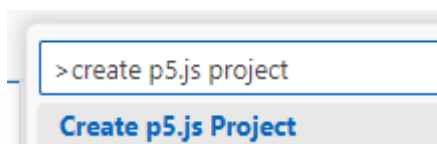
Stejný postup použijeme i pro instalaci rozšíření *Live Server*.



Obrázek 3: rozšíření Live Server

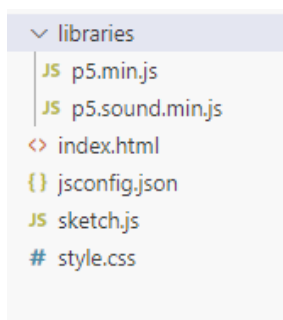
3.1.2 Vytvoření projektu p5.js

Po úspěšné instalaci rozšíření použijeme zkratku **Ctrl+Shift+P**. Otevře se nám horní vyhledávací řádek a předvyplní znak „>“. Tento krok lze udělat i manuálně kliknutím na vyhledávací řádek a napsáním znaku >. Za tento znak napíšeme „**Create p5.js Project**“ a zmáčkeme Enter.



Obrázek 4: vytvoření p5.js projektu

Otevře se Průzkumník souborů a vybereme složku, do které se projekt nainstaluje, případně vytvoříme novou složku. Klikneme na *Vybrat složku* a automaticky se vytvoří soubory *index.html*, *jsconfig.json*, *sketch.js* a *style.css*. Dále se vytvoří podsložka *libraries* obsahující samotné dílčí části knihovny *p5.js* s názvy *p5.min.js* a *p5.sound.min.js*.

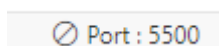


Obrázek 5: vytvořené soubory

3.1.3 Spuštění lokálního serveru

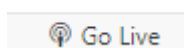
Stejně jako předtím, opět použijeme zkratku **Ctrl+Shift+P** nebo zadáme manuálně a napíšeme

„**Open with Live Server**“. Po kliknutí na klávesu Enter by se měl otevřít výchozí prohlížeč a v něm adresa **127.0.0.1:5500**. Tuto adresu lze také zadat manuálně. Na této adrese se nyní vytvořil lokální server, který bude zobrazovat náš program. Zároveň se na spodní liště ve VSC objevilo **Port : 5500** značící, že náš server je online a běží na portu 5500.



Obrázek 6: Port : 5500

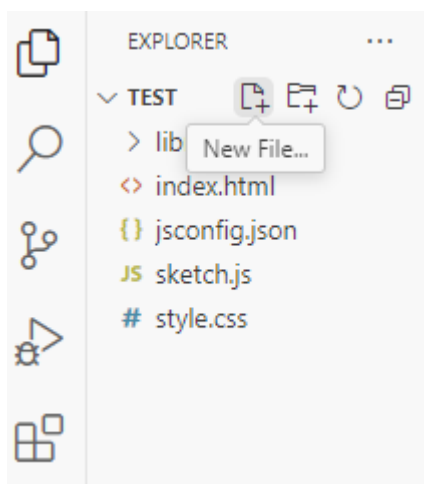
Toto sousloví lze použít jako tlačítko a kliknutím na něj server vypnout. Po vypnutí se změní ikona a napíše se **Go Live**, kliknutím se server opět spustí.



Obrázek 7: Go Live

Nyní máme vytvořený projekt a spuštěný lokální server a můžeme se pustit do psaní programu. Později budeme vytvářet nové soubory, můžeme tak učinit pomocí ikony **New File** v záložce *Explorer* v levé části prostředí (záložka se nachází v horní části lišty a její ikona jsou dvě složky). ⁴

⁴ Popsaný postup přípravy je platný k dubnu 2024, v pozdějších aktualizacích se může lišit



Obrázek 8: tlačítko New File

3.2 Třída Card

3.2.1 Popis třídy a constructor()

Pro správné fungování hry je potřeba adresovat jednotlivé karty, ať už pro zjištění její hodnoty a barvy nebo pro její zobrazení na určité pozici na obrazovce. V jazyce JavaScript pro to existuje vhodné řešení – založení třídy, pomocí které lze vytvářet jednotlivé instance, jednotlivé karty, přičemž každá bude mít vlastní parametry jako například barvu a hodnotu nebo pozici. To umožní pracovat s jednotlivými kartami jako s objekty a přistupovat k těmto parametrům a dále je využívat. Pro třídu jsem vytvořil pro lepší orientaci vlastní soubor *Card.js*, který je ale nutné zahrnout do těla HTML souboru *index.html* pomocí párového tagu `<script>` a uvedením adresy souboru v parametru *src*.

```
18 | | <script src="Card.js"></script>
```

Obrázek 9: zahrnutí souboru Card.js do HTML kódu

Třída se v jazyce JavaScript zakládá klíčovým slovem *class* a celá třída se uzavře do složených závorek. To, jaké bude mít třída parametry, se deklaruje v úvodu třídy metodou **constructor**. Parametry této metody jsou proměnné, které zadáme při zakládání jednotlivých objektů.

```

1  class Card {
2      constructor(suit, value, visible = false) {
3          this.suit = suit;
4          this.value = value;
5          if (suit !== 0) {
6              this.image = loadImage("./images/" + suit + "/card_" + suit + "" + value + ".png");
7          } else {
8              this.image = loadImage("./images/0/card_" + suit + "" + value + ".png");
9          }
10         this.x = 0;
11         this.y = 0;
12         this.visible = visible;
13     }
14 }

```

Obrázek 10: metoda constructor třídy Card

Argumenty objektu třídy *Card* jsou proměnné **suit** (udává barvu karty), **value** (hodnota karty) a **visible** (proměnná typu *boolean*, která udává, zda je karta otočená lícem vzhůru – true, nebo dolů – false). Je možné při zakládání objektu některé parametry nezadat. Například zde, pokud není při deklaraci zadána hodnota proměnné *visible*, automaticky je jí přiřazená hodnota *false* (zapisuje se ve stylu *jmeno_promenne = hodnota_promenne*). Proměnné uvedené v závorkách u slova *constructor* jsou ovšem pouze pomocné, nosné proměnné, pro přiřazení jejich hodnoty vlastním objektovým proměnným se používá klíčové slovo **this**. V prostředí třídy je nutné na vlastní proměnné odkazovat vždy pomocí *this.jmeno_promenne*, pro odkaz na proměnnou již konkrétního objektu se slovo *this* nahradí názvem proměnné, například pro odkaz na hodnotu proměnné *value* objektu s názvem *karta1* se použije *karta1.value*.

Kromě zadávaných proměnných třída *Card* obsahuje i další proměnné, které budou potřeba, a jejichž hodnota je prozatím přiřazena natvrdo. Jako příklad poslouží proměnné **x** a **y**, které budou později určovat pozici karty na obrazovce. Než bude rozhodnuto, jestli se karta bude nacházet v ruce hráče nebo v balíčku, je jim přiřazena hodnota 0. Poslední objektová proměnná je **image**, která se pomocí barvy a hodnoty karty dostane k png obrázku na určité adrese v adresáři stránky, který pomocí metody z knihovny *p5.js* *loadImage()* tento obrázek načte, aby mohl být později použit k zobrazení karty na obrazovce.

3.2.2 show()

První metoda třídy *Card* je metoda **show()**, která bude volána v hlavní metodě *draw*, která se spouští každý snímek. Tato metoda slouží k zobrazení načteného obrázku na obrazovce.

```

15 | show() {
16 |     if (this.visible) {
17 |         image(this.image, this.x, this.y, floor(height / 4 * this.image.width / this.image.height), floor(height / 4));
18 |     } else {
19 |         image(cardback, this.x, this.y, floor(height / 4 * this.image.width / this.image.height), floor(height / 4));
20 |     }
21 | }

```

Obrázek 11: metoda show()

Obrázek je do proměnné *image* načten jako objekt třídy *p5.Image*, který má svoje vlastní proměnné, mezi nimi pro nás nejdůležitější šířka a výška obrázku, jmenovitě *image.width* a *image.height*. Obrázek se zobrazí na obrazovku pomocí p5 metody *image()*, jejíž argumenty jsou postupně: obrázek třídy *p5.Image*, x a y souřadnice obrázku, šířka a výška zobrazení. Aby se karty vešly na jakkoliv vysoký monitor, výšku zobrazení jsem natvrdo nastavil na čtvrtinu obrazovky (proměnná *height* vrátí výšku obrazovky – musí se pod sebe vejít karty v hráčově ruce, karty v ruce soupeře a odhazovací balíček. Pro lepší přehlednost jsem proto nechal ještě nějaké místo mezi nimi, které vytvoří mezeru a lépe oddělí jednotlivé skupiny karet. Potom je nutné zachovat původní poměr obrázku, aby nedošlo k deformaci, proto pro šířku zobrazení jsem vynásobil výšku zobrazení poměrem šířka obrázku ku výšce obrázku: $height / 4 * this.image.width / this.image.height$. Protože výpočet pracuje se zlomky, může se stát, že vyjde desetinné číslo. Výška a šířka se ale udávají v pixelech, tudíž by desetinná hodnota nedávala smysl. Proto je celý výpočet zaobalený do p5 metody *floor()*, která vrátí zadanou hodnotu zaokrouhlenou dolů.

Protože karta může být otočená buď rubem nebo lícem vzhůru, je potřeba zobrazit v každém případě jiný obrázek. Pro otočení lícem vzhůru se použije již načtený obrázek *image*, pro rubovou variantu se použije obrázek *cardback*, který je globální proměnná deklarovaná pro ušetření času a místa pouze jednou v hlavní metodě *preload()*, o které bude řeč později. To, který obrázek se bude zobrazovat, závisí hodnota proměnné *visible*.

3.2.3 setPos(), flip()

Při „líznutí“ karty nebo jejím odehrání se logicky mění poloha karty. K možnosti změnit tuto polohu dynamicky v průběhu hry slouží jednoduchá metoda **setPos()**, jejíž parametry jsou souřadnice *x* a *y*. Dosavadní souřadnice karty tak dostanou novou hodnotu.

```
23 |     setPos(x, y) {  
24 |         this.x = x;  
25 |         this.y = y;  
26 |     }
```

Obrázek 12: metoda setPos()

Podobně funguje i metoda **flip()**, která bude volána při otočení karty. Její jediná funkce je obrácení hodnoty proměnné *visible* pomocí negace. Ta se značí pomocí znaku “!” a lze použít na proměnné typu *boolean*. Z hodnoty *true* se po negaci stane *false* a naopak.

```
28 |     flip() {  
29 |         this.visible = !this.visible;  
30 |     }
```

Obrázek 13: metoda flip()

3.2.4 clicked()

Pro vykonání herního tahu je nutné detekovat, jestli v daném snímku hráč kliknul na určitou kartu. K detekci kliknutí myši kdekoliv na obrazovce se využívá p5 metoda *mouseClicked()* v hlavním kódu a až uvnitř této metody se zkontroluje, kde přesně bylo kliknuto pomocí objektové metody **clicked()**, protože logicky je nutné kontrolovat lokaci kurzoru pouze v moment, kdy bylo kliknuto.

```
32 |     clicked() {  
33 |         if (mouseX >= this.x - floor(height / 4 * this.image.width / this.image.height) / 2  
34 |             && mouseX <= this.x + floor(height / 4 * this.image.width / this.image.height) / 2) {  
35 |             if (mouseY >= this.y - height / 4 / 2 && mouseY <= this.y + height / 4 / 2) {  
36 |                 return true;  
37 |             }  
38 |         }  
39 |  
40 |         return false;  
41 |     }
```

Obrázek 14: metoda clicked()

K určení lokace kurzoru slouží globální proměnné knihovny p5 **mouseX** a **mouseY**. Pokud hráč klikne na kartu, kurzor musí být umístěn mezi hranicemi zobrazené karty. K tomu jsem využil určení hodnot, mezi kterými musí kurzor stát, aby byla událost vyhodnocena jako kliknutí na kartu. Obrázek díky metodě *image()* bude mít střed přesně na souřadnicích *this.x* a *this.y*, tudíž pro určení spodní a horní hranice je potřeba k těmto souřadnicím přičíst nebo odečíst půlku šířky nebo výšky obrázku. Výrok *if* zkontroluje, zda hodnota *mouseX* je v tomto oboru a poté další vnořené *if* zkontroluje to samé pro hodnotu *mouseY*. Pro logický součin se používá operátor **&&**. Pokud kurzor uspěje v obou případech, metoda vrátí hodnotu *true* (bylo kliknuto na kartu), pokud se ale aspoň jedna ze souřadnic kurzoru nachází mimo hranice, vrátí se hodnota *false* (bylo kliknuto mimo kartu). Vracená hodnota bude použita k zavolání dalších metod, například líznutí nebo odehrání karty.

3.2.5 use()

Pro samotné použití karty jsem vytvořil metodu **use()**, která je volána v hlavní části kódu a používá metodu *useCard()*, o které budu psát až v části věnující se přímo hlavnímu kódu. Zavolání metody způsobí to, že se karta přemístí z ruky hráče do odhazovacího balíčku.

```
43     use() {  
44         |     hand1 = useCard(hand1, getIndex(this, hand1));  
45     }  
46 }
```

Obrázek 15: metoda use()

3.3 Třída Button

3.3.1 Popis třídy a constructor()

Pro lepší ovladatelnost a funkcionalitu je uživateli umožněno například pozastavit hru, restartovat nebo ovládat zvukové efekty. Řešení, které jsem se rozhodl použít, je zprostředkovat to pomocí tlačítek. Jazyk HTML sice umožňuje využití přednastavených tlačítek, ale ta nevyhovují zcela tomuto řešení. Rozhodl jsem se proto vytvořit vlastní třídu, která by obsahovala všechny potřebné metody a splňovala moje požadavky. Třidu jsem pojmenoval **Button**. Výsledný objekt této třídy bude tlačítko, které lze aktivovat a deaktivovat (při kliknutí na deaktivované tlačítko se nic nestane), lze nastavit vlastní ikonu tohoto tlačítka a metodu, která bude zavolána, pokud bude na tlačítko kliknuto.

Také tuto třídu je potřeba zmínit v souboru *index.html* tagem `<script>`, jako tomu bylo u třídy *Card*.

Metoda *constructor()* obsahuje souřadnice, na kterých bude tlačítko zobrazeno (**x** a **y**), adresu, ze které si tlačítko načte obrázek pomocí již zmiňované metody *loadImage()*. Dále je zde proměnná **active**, která určuje, zda je tlačítko aktivní nebo ne, tato proměnná je přednastavená na hodnotu *false*. U některých tlačítek (například tlačítko hlasitosti) je potřeba zaznamenat informaci o tom, v jakém stavu se tlačítko nachází (plná hlasitost/ztišeno). K tomu slouží proměnná **id**, do které lze uložit jakýkoliv textový řetězec, podle kterého půjde později uložená informace zjistit. Proměnná je přednastavena na prázdný textový řetězec – nemá žádný identifikátor.

```
1  class Button {
2      constructor(x, y, imageLink, active = false, id = "") {
3          this.x = x;
4          this.y = y;
5          this.img = loadImage(imageLink);
6          this.active = active;
7          this.id = id;
8      }
```

Obrázek 16: metoda *constructor()* třídy *Button*

3.3.2 *setFunction()*, *setImage()*, *activate()*

Před kliknutím na tlačítko je ještě potřeba určit, co se má v takovém případě stát. Jazyk JavaScript umožňuje jako argument metody uvádět jiné metody a poté je volat. Toho jsem využil v tomto případě, kdy jsem vytvořil metodu *setFunction()*, která do nové objektové proměnné **clickedFunction** uloží metodu, která bude volána pokaždé, když uživatel klikne na toto tlačítko. Mohlo by se zdát, že proměnnou *clickedFunction* šlo deklarovat už v metodě *constructor*, ale v některých případech je nutné odkazovat se přímo na již existující tlačítko, které by v tomto případě ještě neexistovalo. Proto je potřeba to řešit přes separátní metodu.

```
10     setFunction(f) {
11         this.clickedFunction = f;
12     }
```

Obrázek 17: metoda *setFunction()*

Podobně funguje i metoda *setImage()*, která změní zobrazovaný obrázek. Metoda jako svůj argument bere již načtený objekt *p5.Image()* a nastaví jej jako hodnotu objektové proměnné *img*. To se hodí v případech, kdy potřebujeme jinou ikonu pro zapnutý a vypnutý stav (například již zmiňované tlačítko hlasitosti).

```
37 |     setImage(image) {  
38 |         |     this.img = image;  
39 |         |  
40 |     }
```

Obrázek 18: metoda *setImage()*

Metodou *activate()* je možné přehodit stav proměnné *active* z vypnuté na zapnutou a naopak (opět se zde střídají hodnoty *true* a *false*).

```
33 |     activate() {  
34 |         |     this.active = !this.active;  
35 |         |  
    |     }
```

Obrázek 19: metoda *activate()*

3.3.3 *show()*, *clicked()*

Stejně jako u třídy *Card*, i tady je potřeba tlačítko v každém snímku zobrazit na obrazovce. V tomto případě ale program nejdříve zkontroluje, jestli je tlačítko aktivní, a pouze potom jej zobrazí, opět pomocí *p5* metody *image()*.

```
27 |     show() {  
28 |         |     if (this.active) {  
29 |         |         |     image(this.img, this.x, this.y);  
30 |         |         |  
31 |         |     }
```

Obrázek 20: metoda *show()*

Také metoda *clicked()* funguje obdobně jako u třídy *Card*. Metoda bude opět volána pouze po kliknutí myši kamkoliv na obrazovku a pomocí souřadnic kurzoru a hranic obrázku určí, zda bylo kliknuto na tlačítko. V takovém případě zavolá načtenou metodu z proměnné *clickedFunction()*, jejíž argumentem bude toto tlačítko. To je potřeba k tomu, aby bylo možné z této metody ovládat například deaktivaci tlačítka nebo změnu jeho stavu.

Metoda *clicked()* zároveň slouží jako detektor, zda bylo kliknuto na tlačítko a vrací hodnoty *true* – bylo kliknuto, nebo *false* – bylo kliknuto jinam.

```
14     clicked() {
15         if (this.active) {
16             let dx = abs(this.x - mouseX);
17             let dy = abs(this.y - mouseY);
18             if (dx <= this.img.width / 2 && dy <= this.img.height / 2) {
19                 this.clickedFunction(this);
20                 return true;
21             }
22         }
23
24         return false;
25     }
```

Obrázek 21: metoda *clicked()*

3.4 Hlavní část *sketch.js*

3.4.1 Základní proces hry

Nyní se dostáváme k samotné hlavní části programu, která bude ovládat celý herní cyklus. Tato část je uložena v samostatném souboru *sketch.js*, který je opět potřeba uvést v *index.html* tagem `<script>`. Program se dělí na tři hlavní části, metody **preload** (slouží k deklaraci nebo přednačení důležitých proměnných, jako jsou třeba obrázky, zvuky, fonty, ještě před samotným zobrazením stránky v prohlížeči), **setup** (vytvoření zobrazovacího plátna, deklarace proměnných, které musí existovat před spuštěním herního cyklu atd.) a **draw** (metoda, která je volána každý snímek znovu, v tomto případě slouží jako herní cyklus, je v ní skryto například znovuzobrazení všech obrázků nebo detekce změny stavu hry, restart nebo konec hry).

Hra se může nacházet ve třech základních stavech – úvodní/pause menu, samotná hra nebo závěrečná obrazovka. K rozlišení těchto stavů slouží proměnná **gamestate**, která může nabývat tří hodnot (menu, hra nebo závěr). Na základě toho, jakou má proměnná hodnotu se zobrazí příslušná obrazovka, tlačítka a volají příslušné metody.

K monitorování určitých stavů již uvnitř hry slouží další „stavové proměnné“, které uchovávají například informaci o tom, jaký hráč je zrovna na tahu, jestli bylo položeno eso nebo sedmička, jestli je stále aktivní (po líznutí nebo přeskočení tahu se efekt karty deaktivuje) nebo jestli se mění znak karty při zahrání dámy. Hodnoty těchto

proměnných se mění určitými karetními akcemi a na základě nich se kontroluje, zda je možné určité karetní úkony v souvislosti s pravidly provést. Například při aktivním esu je proti pravidlům líznout si kartu či zahrát jinou kartu než eso. V takových případech program nedovolí tyto akce učinit a dovolí pouze akce v souladu s pravidly.

3.4.2 Balíčky a karetní akce

Pomocí třídy *Card* již máme vytvořené formy karet, teď je ale potřeba je seskupit do jednotlivých skupin, které mezi sebou mohou interagovat (odhazovací a dobírací balíček a karty, které mají v ruce oba hráči). K tomu jsem zvolil řešení pomocí polí. **Pole** je uspořádaná řada jednotlivých objektů, každá pozice je indexována (označena číslem) a jednotlivé objekty se dají do pole přidávat, odstraňovat, třídit a mnoho dalších funkcí. V tomto případě je pro každou skupinu karet založeno vlastní pole, do kterého jsou přidány jednotlivé karty třídy *Card*. Tyto proměnné jsou deklarovány v horní části kódu a nesou názvy *deck* (dobírací balíček), *gamedeck* (odhazovací balíček), *hand1* a *hand2* (ruka uživatele a ruha protivníka). Pomocí následujících metod je možné například balíček zamíchat, líznout si kartu, zahrát kartu atd.

3.4.2.1 *shuffleDeck()*

Tato metoda slouží k úvodnímu zamíchání dobíracího balíčku před každou hrou. Míchání karet je v tomto případě řešeno náhodným zvolením dvou karet v rámci pole a vyměněním jejich pozic. Argumentem této metody se uvádí počet výměn.

```
327 function shuffleDeck(iterations) {
328     for (let i = 0; i < iterations; i++) {
329         let a = floor(random(0, deck.length));
330         let b = floor(random(0, deck.length));
331         while (b == a) {
332             b = floor(random(0, deck.length - 1));
333         }
334
335         let hold = deck[a];
336         deck[a] = deck[b];
337         deck[b] = hold;
338     }
339 }
```

Obrázek 22: metoda *shuffleDeck()*

3.4.2.2 *drawCard()*, *useCard()*

Metody *drawCard()* a *useCard()* jsou volány pokaždé, když hráč klikne na kartu (*drawCard()*) v případě kliknutí na dobírací balíček a *useCard()* pokud klikne na kartu ve svém balíčku.

Při líznutí se musí specifikovat, do jaké ruky se má karta přidat, jestli se má otočit a kolik karet se vůbec dobírá. Uvnitř metody se poté zkontroluje, zda je tento tah legální a poté se vrchní karta z pole dobíracího balíčku přesune do pole zadané ruky. Metoda *drawCard()* je také volána při použití tlačítka na líznutí více karet najednou, pokud protihráč zahraje sedmičku (viz kapitola 2.2.2).

Metoda *useCard()* je o něco složitější, při pokusu o odehrání karty se nejdříve zkontroluje, jestli je možné zvolenou kartu zahrát a poté, zda daná karta vyvolává nějaký speciální efekt. Pokud ano, zavolají se příslušné metody a případně aktivují určitá tlačítka (například tlačítko k přeskočení tahu či tlačítka výběru změny znaku). V závislosti na efektu dané karty se také změní hodnota příslušných *stavových proměnných*.

3.4.3 *mouseClicked()*

Tato metoda frameworku *p5* je volána pokaždé, když uživatel klikne kurzorem na obrazovku. Uvnitř této metody jsou v závislosti na stavu proměnné *gamestate* volána metoda *clicked()* jednotlivých tlačítek.

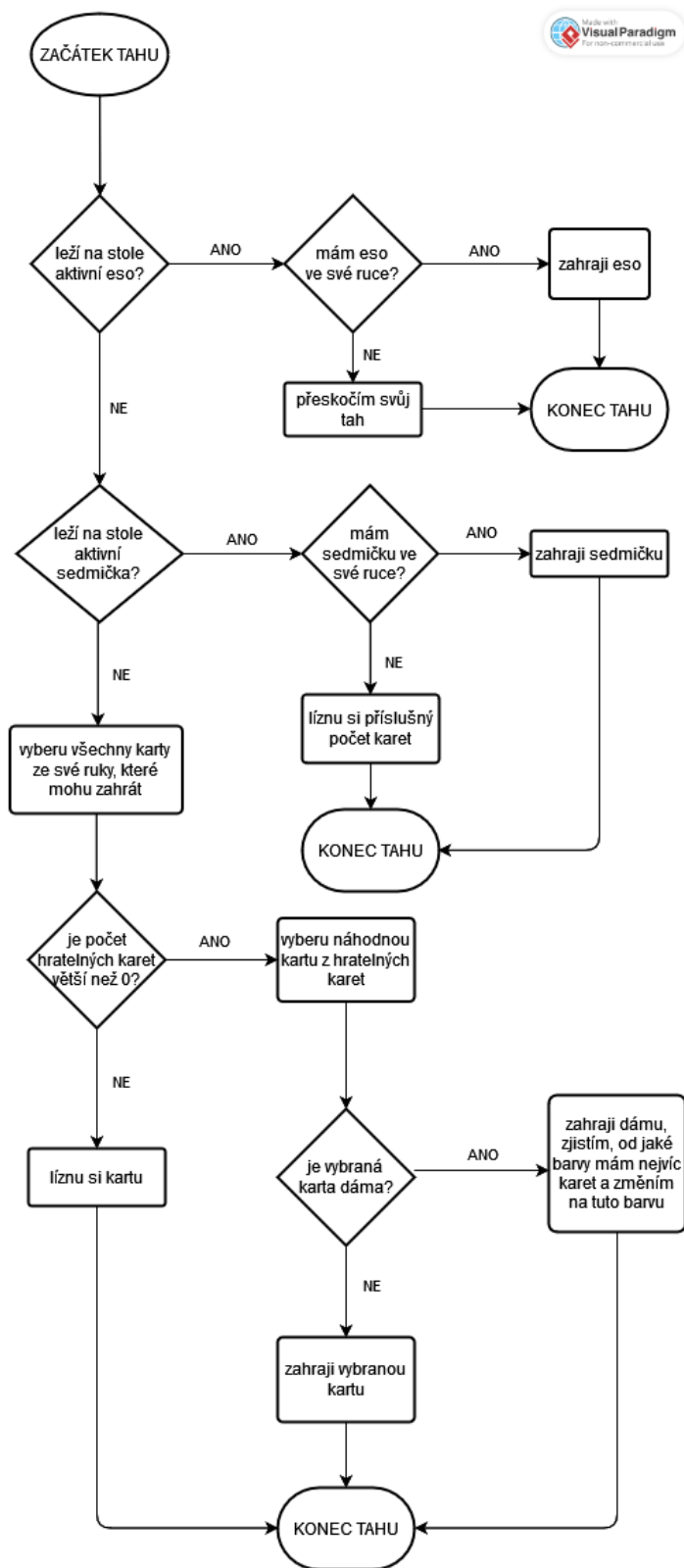
Metoda *clicked()* je také volána u jednotlivých karet v ruce hráče. Jelikož se karty překrývají, mohlo by se stát, že jedním kliknutím se zaregistruje kliknutí na dvou kartách současně. Proto se tato metoda volá postupně ve vlně tak, jak jsou po sobě karty seřazeny v balíčku. Nejdříve se zkontroluje vrchní karta, poté karta pod ní atd. V moment, kdy je zaregistrováno kliknutí na některou z karet, pomocí příkazu *break* se ukončí kontrolní cyklus a další karty již kontrolovány nejsou. Tím je zaručeno, že uživatel vždy klikne pouze na jedinou kartu.

3.4.4 Protihráč

3.4.4.1 Herní logika

Protihráč je ve hře naprogramován jako velmi primitivní „umělá inteligence“, tedy algoritmus sestavený z několika logických kroků na základě nichž se program rozhodne pro některý z dostupných herních úkonů. Tuto logiku znázorňuje diagram na následující stránce.

Na začátku tahu protihráče program nuceně počká 100 snímků. Tím je simulovaný přirozenější vzhled algoritmu – hráč se přeci také nerozhodne během zlomku vteřiny, ale nějaký čas přemýšlí a uvažuje nad možným krokem. Kdyby zde tato prodleva nebyla, uživatel by ihned po odehrání byl opět na tahu, jelikož celkový výpočet algoritmu trvá řádově několik milisekund, a bylo by složité orientovat se v průběhu hry. K lepší orientaci ve hře pomáhá také *tahový token*. Je to žeton nacházející se mimo hru, který mění svoji barvu v závislosti na tom, který hráč je zrovna na tahu.



Obrázek 23: diagram algoritmu protihráče⁵

⁵ VisualParadigm. Online. 2024. Dostupné z: <https://online.visual-paradigm.com/>. [cit. 2024-04-09].

3.4.5 Konec hry, restart

Přírozně podle pravidel hra končí ve chvíli, kdy jeden z hráčů odehraje svoji poslední kartu. Pro větší jednoduchost hry jsem neuvažoval o možnosti vrácení do hry, tedy možnosti protihráče zahrát sedmičku a přimět tím vítězného hráče líznout si další karty a pokračovat ve hře.

Ihned po konci hry se změní herní stav na 1, tedy závěrečná obrazovka. Z té se uživatel může tlačítkem dostat opět na úvodní obrazovku a hrát znovu. Stisknutím tohoto tlačítka se všechny *stavové proměnné* vrátí zpět na původní hodnotu, všechny karty se vrátí do dobíracího balíčku a ten je zamíchán. Hráč poté může zapnout hru a hrát od začátku.

Tlačítko restartu se nachází také v rohu obrazovky při hraní hry. Toto tlačítko ovšem pro zrychlení přeskočí úvodní obrazovku a pouze uvede stavové proměnné do původního stavu.

3.4.6 Grafická podoba

Veškeré obrázky jsou vytvořeny pomocí programu *Malování* dostupného v operačním systému Windows. Jelikož se nepokládám za příliš výtvarně nadaného, rozhodl jsem se pro minimalistický design známý jako *pixel-art*, ve kterém se zredukováním rozlišení na jednotlivé pixely navozuje efekt starých retro arkádových her. Obrázky jsem tedy vytvářel pomocí pixelové mřížky a poté pomocí programu *Malování 3D* zvětšil a odstranil pozadí, jelikož program *Malování* neumožňuje ukládat obrázky s transparentním pozadím.



Obrázek 24: grafická podoba karet ve hře

3.4.7 Zvukové efekty a hudba

Podobně jako obrázky, i zvukové soubory jsou do programu nahrávány pomocí metody knihovny *p5*. Jedná se o metodu *loadSound()*, která je volána v rámci metody *preload()* ještě před načtením stránky, aby bylo možné zvukové efekty využít hned od první chvíle.

```
47 | menusong = loadSound("./sounds/menu.mp3");
```

Obrázek 25: metoda *loadSound()*

Jednotlivé zvukové efekty se poté spouští v rámci jiných metod, například zvuk kliknutí se spustí pokaždé při kliknutí na tlačítko a zvuk položení karty při každé odehrané kartě. Zvukové efekty líznutí a položení karty jsem nahrál pomocí reálných karet, zbytek zvuků je vytvořených stránkou *sfxr.me*⁶.

Ve hře je také možné slyšet dvě delší skladby – první v počátečním, pause a závěrečném



Obrázek 26: tlačítka ovládání zvuku

menu a druhou při hře. Obě jsem vytvořil v programu *FL Studio 21* a nahrál do programu opět pomocí metody *loadSound()*. Tyto skladby jsou nastavené tak, aby se po skončení znovu spustily a hrály donekonečna. Jejich hlasitost se dá změnit, resp. dají se úplně ztlumit pomocí tlačítka, stejně tak ostatní zvukové efekty.

⁶ *Jsfxr*. Online. 2024. Dostupné z: <https://sfxr.me/>. [cit. 2024-04-09].

3.4.8 Textový font

Aby hra vypadala celkově ještě o trochu lépe, místo základního textového fontu jsem na stránce *fontstruct.com*⁷ vytvořil vlastní „pixelový“ font, podobný například fontu ve starých arkádových hrách. Font je uložený s příponou *.ttf* a načtený do programu metodou knihovny *p5 loadFont()*. Ten je poté nastaven jako výchozí metodou *textFont()*.

3.5 Nahrávání na server

Aby mohla být hra spuštěna online, zaregistroval jsem na webu *webzdarma.cz* doménovou adresu třetího řádu **kroulitsch.wz.cz**, na kterou jsem přímo pomocí administračních nástrojů webu nahrál všechny potřebné soubory – HTML soubor *index.html* a konfigurační soubory, hlavní kód *sketch.js*, třídivé soubory (*Card.js* a *Button.js*), knihovnu *p5.js*, font, všechny zvukové soubory a obrázky. Zachoval jsem hierarchii a rozdělení jednotlivých souborů do příslušných složek, stejně tak, jak na ně odkazují metody v kódu. Na této stránce je hra plně k dispozici a přístupná komukoliv, kdo stránku spustí.

⁷ FontStruct. Online. 2024. Dostupné z: <https://fontstruct.com/>. [cit. 2024-04-09].

4 Závěr

V této práci byl vytvořen kód v programovacím jazyce JavaScript, který zpracovává karetní hru prší v internetovém prohlížeči. Hra obsahuje vlastní obrázky, zvukové efekty, hudbu a textový font. Celý program je vložený na webové stránky a je volně dostupný. Hlavní náplní hry je partie karetní hry prší proti počítačovému algoritmu. Hra je řízena podle předem určených pravidel včetně dobírání a odehrání karty, přeskočení tahu či změny znaku karty. Program obsahuje ovládací prvky ve formě tlačítek, která umožňují uživateli pozastavit hru, restartovat, ovládat hlasitost a interagovat s herním prostředím.

Za zmínku stojí možná vylepšení, která by mohla hru posunout na vyšší úroveň. Jedním z nich je například zvýšení úrovně „přemýšlení“ oponenta, který by kupříkladu mohl pomocí složitějších algoritmů kalkulovat nejlepší možný tah na několik kol dopředu. Další možné zlepšení by mohlo být umožnění dvěma uživatelům připojit se na server a hrát online proti sobě. Jedním z vylepšení, která mi navrhli uživatelé, kteří hru vyzkoušeli, a které jsem zrealizoval, bylo vytvoření verze pro mobilní telefon a menší obrazovky. Do HTML kódu stránky jsem přidal metodu, která na základě daného zařízení přesměruje uživatele buďto na mobilní, nebo počítačovou verzi. Mobilní verze je jednodušší, jednotlivé elementy jsou přizpůsobeny velikosti i proporcím displeje. Za další nedostatek považuji někdy zbytečnou složitost celého programu. Vyskytují se zde opakující se části, které by bylo možné nahradit zavedením obecnějších metod a tím by došlo ke zjednodušení orientace v kódu.

Odkaz na webovou stránku s finální verzí hry:

<http://kroulitsch.wz.cz>

5 Seznam zdrojů

1. *Programovací jazyky*. Online, prezentace. Praha: ČVUT, 2017. Dostupné z: https://www.fd.cvut.cz/personal/jerabem1/14prg/14PRG_03_programovaci_jazyky.pdf. [cit. 2024-04-09]
2. *Jak se hraje prší?* Online. Justit.cz. 2023. Dostupné z: <https://justit.cz/2023/05/10/jak-se-hraje-prsi/>. [cit. 2024-04-09]
3. *VisualParadigm*. Online. 2024. Dostupné z: <https://online.visual-paradigm.com/>. [cit. 2024-04-09]
4. *sfxr*. Online. 2024. Dostupné z: <https://sfxr.me/>. [cit. 2024-04-09].
5. *FontStruct*. Online. 2024. Dostupné z: <https://fontstruct.com/>. [cit. 2024-04-09].

6 Seznam obrázků

Obrázek 1: levý sloupec se záložkami ve VSC	9
Obrázek 2: rozšíření p5.vscode	10
Obrázek 3: rozšíření Live Server	10
Obrázek 4: vytvoření p5.js projektu	10
Obrázek 5: vytvořené soubory	11
Obrázek 6: Port : 5500.....	11
Obrázek 7: Go Live	11
Obrázek 8: tlačítko New File.....	12
Obrázek 9: zahrnutí souboru Card.js do HTML kódu	12
Obrázek 10: metoda constructor třídy Card	13
Obrázek 11: metoda show()	14
Obrázek 12: metoda setPos()	15
Obrázek 13: metoda flip()	15
Obrázek 14: metoda clicked().....	15
Obrázek 15: metoda use()	16
Obrázek 16: metoda constructor() třídy Button	17
Obrázek 17: metoda setFunction()	17
Obrázek 18: metoda setImage()	18
Obrázek 19: metoda activate()	18
Obrázek 20: metoda show()	18
Obrázek 21: metoda clicked().....	19
Obrázek 22: metoda shuffleDeck()	20
Obrázek 23: diagram algoritmu protihráče.....	23
Obrázek 24: grafická podoba karet ve hře	25
Obrázek 25: metoda loadSound().....	25
Obrázek 26: tlačítka ovládání zvuku	25